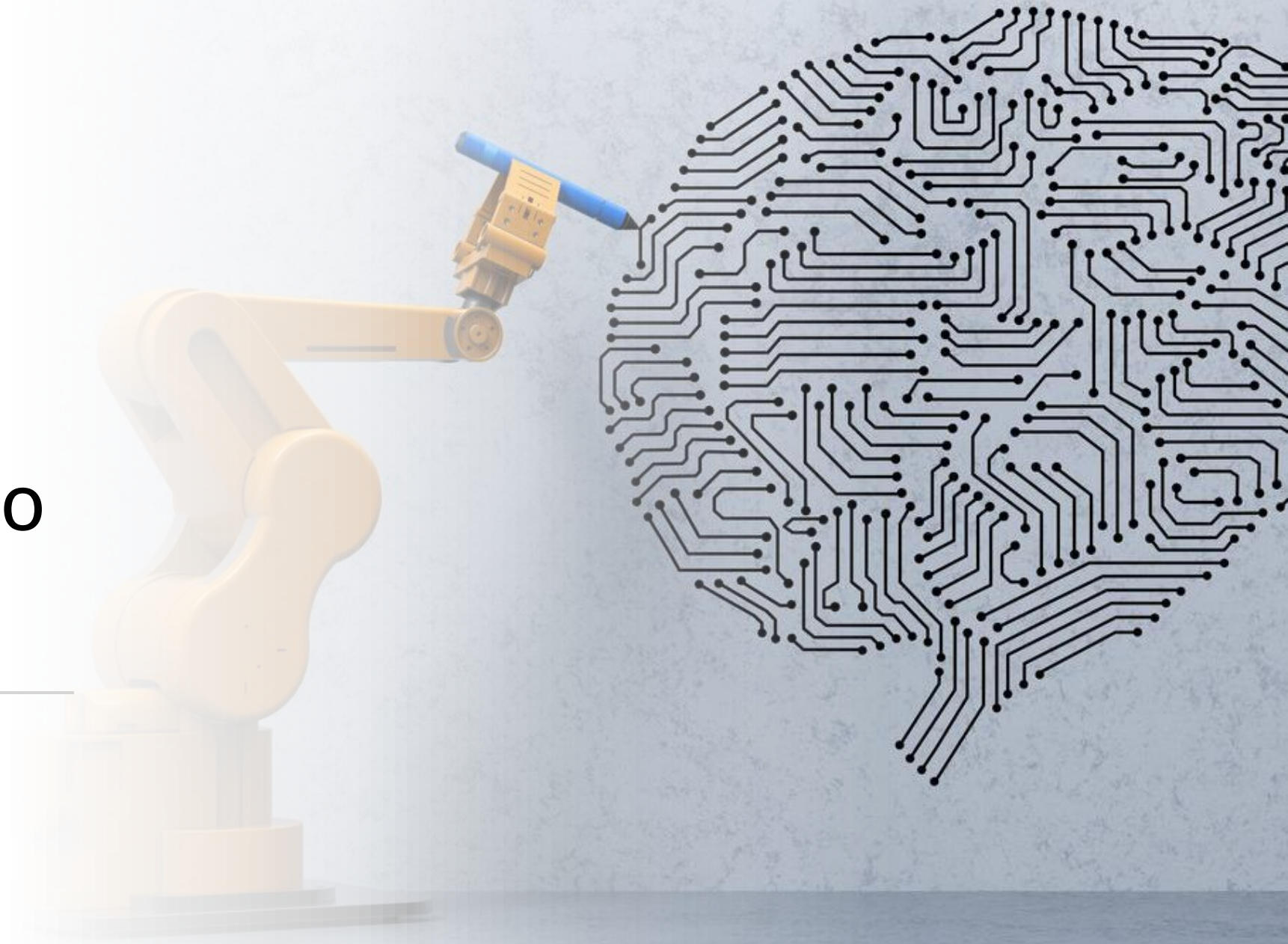
Reconocimiento de Patrones



Clase 11

Clasificadores de Aprendizaje Automático

Antes de empezar

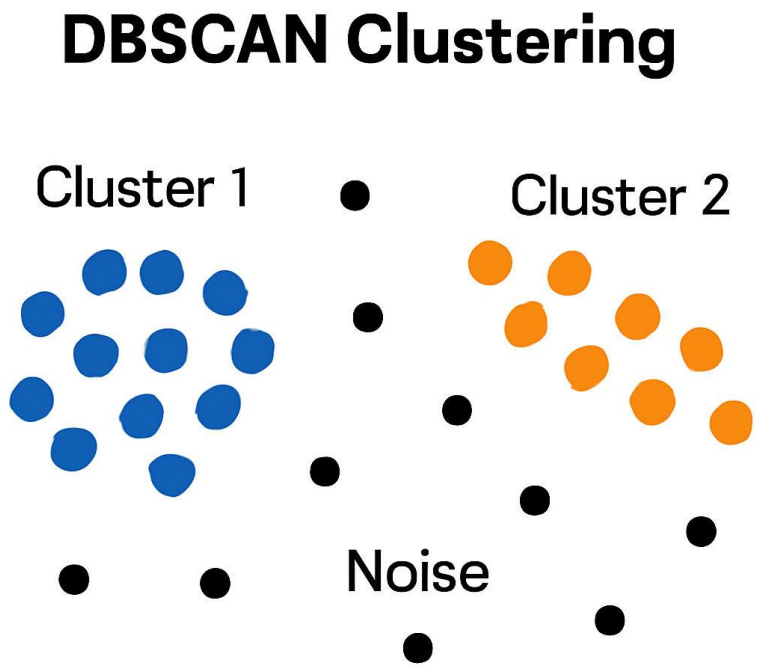
- ¿Dudas de la clase anterior?



DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Es un algoritmo de clustering no supervisado basado en densidad. Se distingue por no requerir que se especifique el numero de clústeres y su capacidad de identificar ruido.

El algoritmo trabaja definiendo los clústeres como regiones densas que están separadas por regiones de menor densidad.



El algoritmo trabaja definiendo únicamente 2 parámetros:

- **Epsilon** ($\boldsymbol{\varepsilon}$). La distancia máxima permitida para que dos puntos se consideren vecinos.
- **Puntos mínimos (MinPts)**. El numero mínimo de puntos que deben de estar dentro del radio ϵ para formar una región densa.

A partir de estos dos parámetros se pueden definir los siguientes conceptos clave.

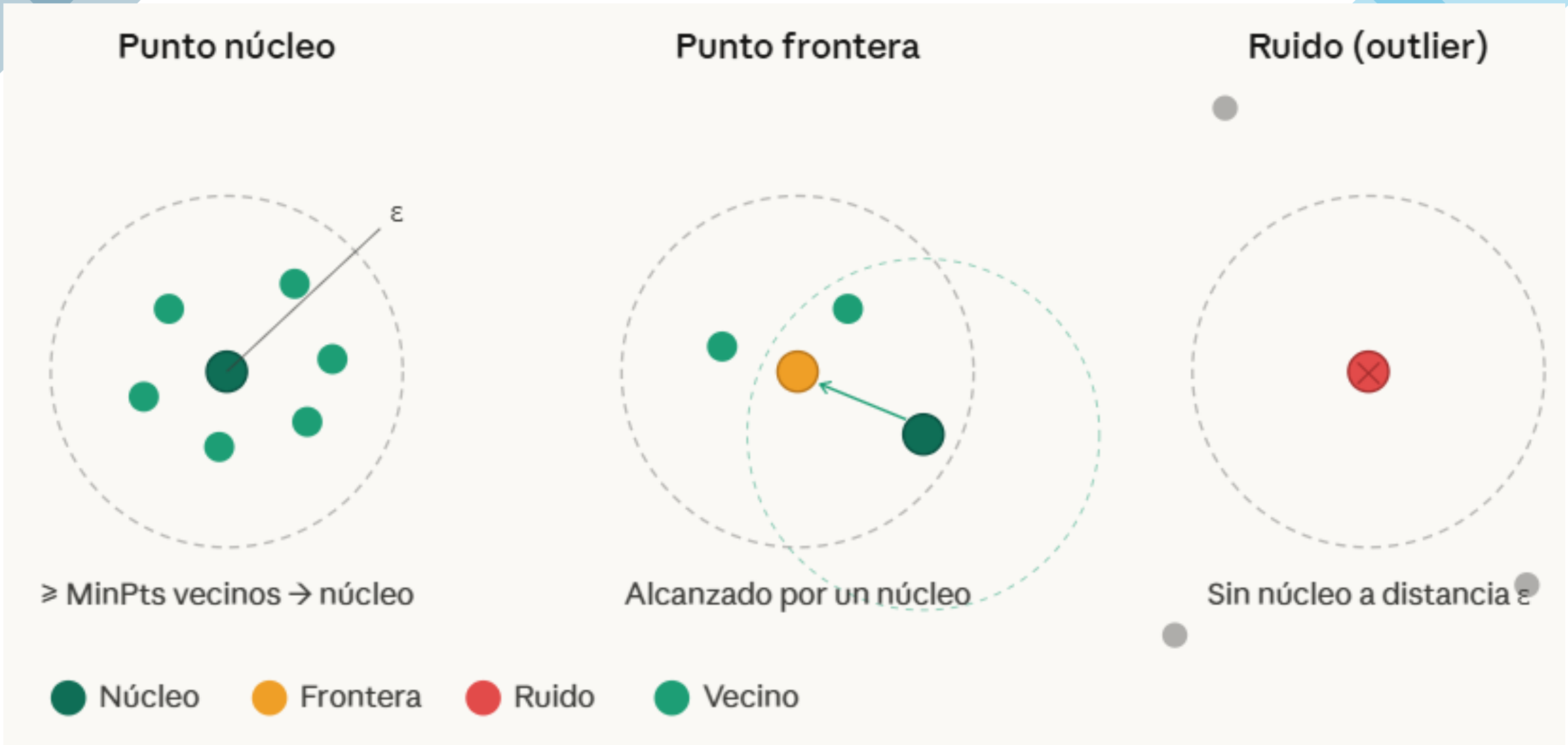
1. Punto Núcleo. Es aquel que tiene al menos MinPts puntos dentro de su vecindad ε, es decir, tienen al menos un número mínimo de otros puntos dentro de una distancia especificada.

$$| N_{\varepsilon}(x) | \geq MinPts$$

2. Punto Frontera. Se encuentra dentro de la distancia ε de un punto central, pero no tienen el mínimo de puntos requeridos MinPts.

3. Ruido. No son puntos núcleo ni frontera ya que no están lo suficientemente cerca de un cumulo para ser incluidos.




Punto núcleo
Punto frontera
Ruido (outlier)
ε
≥ MinPts vecinos → núcleo
Alcanzado por un núcleo
Sin núcleo a distancia ε
Núcleo
Frontera
Ruido
Vecino

DBSCAN

Parámetros y notación

Símbolo	Descripción
D	Conjunto de datos: $D = \{p_1, p_2, \ldots, p_n\}$
ε	Radio de vecindad (distancia máxima entre vecinos)
$MinPts$	Número mínimo de puntos para formar una región densa
$\mathcal{N}_\varepsilon(p)$	Vecindad ε de p: $\{q \in D \mid \text{dist}(p,q) \leq \varepsilon\}$
RUIDO	Etiqueta especial para puntos no asignados a ningún clúster
NoVisitado	Estado inicial de todos los puntos

Algorithm 1: `DBSCAN`(D, ε, $MinPts$)

Entradas:
- D — conjunto de n puntos en un espacio métrico
- $\varepsilon \in \mathbb{R}^+$ — radio de vecindad
- $MinPts \in \mathbb{Z}^+$ — densidad mínima de un punto núcleo

Salidas:
- $etiquetas[1 \ldots n]$ — asignación de clúster para cada punto: $etiquetas[i] \in \{1, 2, \ldots, k\} \cup \{\text{Ruido}\}$
- k — número de clústeres encontrados

Inicialización: $etiquetas[i] \leftarrow$ NoVisitado para todo $p_i \in D$;
$k \leftarrow 0$; ▷ contador de clústeres

1 **para cada** *punto* $p \in D$ **hacer**
2 **si** $etiquetas[p] \neq$ *NoVisitado* **entonces**
3 **continuar** ; ▷ ya fue procesado
4 $N \leftarrow$ `Vecinos`(p, ε, D) ; ▷ puntos dentro del radio ε
5 **si** $|N| < MinPts$ **entonces**
6 $etiquetas[p] \leftarrow$ Ruido ; ▷ p es punto de ruido (puede cambiar después)
7 **si no**
8 $k \leftarrow k + 1$; ▷ nuevo clúster
9 `ExpandirCluster`$(p, N, k, \varepsilon, MinPts, D, etiquetas)$;
10 **retornar** $etiquetas$, k

Algorithm 2: `ExpandirCluster`(p, N, k, ε, $MinPts$, D, $etiquetas$)

Entradas:
- p — punto núcleo semilla del clúster
- N — vecindad ε inicial de p
- k — identificador del clúster actual
- ε, $MinPts$, D, $etiquetas$ — parámetros globales

Salidas:
- $etiquetas$ actualizado: todos los puntos alcanzables desde p quedan asignados al clúster k

1 $etiquetas[p] \leftarrow k$;
2 $Cola \leftarrow N$; ▷ semilla de expansión
3 **mientras** $Cola \neq \emptyset$
4 $q \leftarrow Cola$.extraer();
5 **si** $etiquetas[q] =$ *Ruido* **entonces**
6 $etiquetas[q] \leftarrow k$; ▷ punto frontera: reclasificado
7 **si** $etiquetas[q] \neq$ *NoVisitado* **entonces**
8 **continuar** ; ▷ ya pertenece a un clúster
9 $etiquetas[q] \leftarrow k$;
10 $N' \leftarrow$ `Vecinos`(q, ε, D);
11 **si** $|N'| \geq MinPts$ **entonces**
12 $Cola \leftarrow Cola \cup N'$; ▷ q es núcleo: expandir

Algorithm 3: `Vecinos`(p, ε, D)

Entradas:
- p — punto de consulta
- ε — radio de vecindad
- D — conjunto de datos

Salidas:
- $\mathcal{N}_\varepsilon(p) = \{\, q \in D \mid \text{dist}(p, q) \leq \varepsilon \,\}$

1 **retornar** $\{\, q \in D \mid \text{dist}(p, q) \leq \varepsilon \,\}$

Paso a paso

1. Inicialización. Se marcan todos los puntos del conjunto D como *no visitados* y se establece el contador de clústeres k=0.
2. Iterar sobre cada punto. Para cada punto *p* en D:
   - Si ya fue visitado se pasa por alto y continua al siguiente.
   - Se encuentran todos los puntos $q \in D$ que se encuentren a una distancia menor o igual a ε de *p*. A este conjunto de puntos se le llama $\mathcal{N}_\varepsilon(p)$.



3. Clasificar el punto.

- Si $|\mathcal{N}_\varepsilon(p)| < \mathrm{MinPts}$ se marca *p* como **ruido**
- Si $|\mathcal{N}_\varepsilon(p)| \geq \mathrm{MinPts}$ se etiqueta *p* como **punto núcleo.** k se incrementa e inicia la expansión del clúster

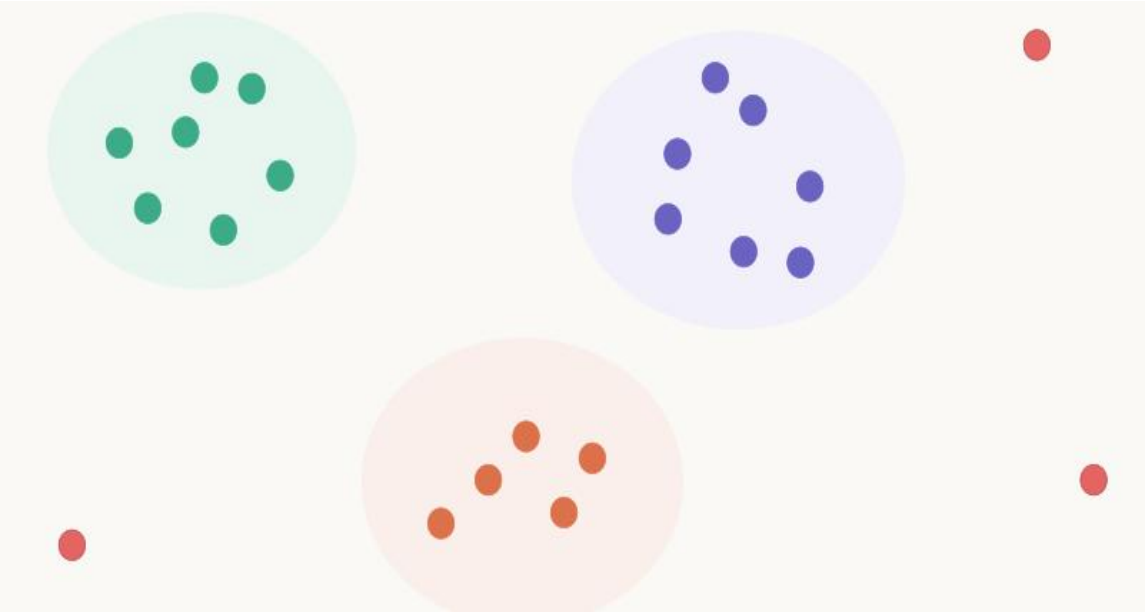
4. Expandir el clúster. (Solo cuando *p* es un núcleo)

Agregar todos los puntos de $\mathcal{N}_\varepsilon(p)$a una cola de expansión y asigna p al clúster k. Luego, para cada punto q en la cola:

- Si *q* estaba marcado como ruido: reclasificarlo al clúster k (es un punto frontera)
- Si *q* ya pertenece a un clúster: ignorarlo
- Si *q* es *no visitado*: asignar al clúster k, calcular su vecindad $\mathcal{N}_\varepsilon(p)$, y si se cumple $|\mathcal{N}_\varepsilon(p)| \geq \mathrm{MinPts}$, añadir estos vecinos a la cola para seguir expandiendo.

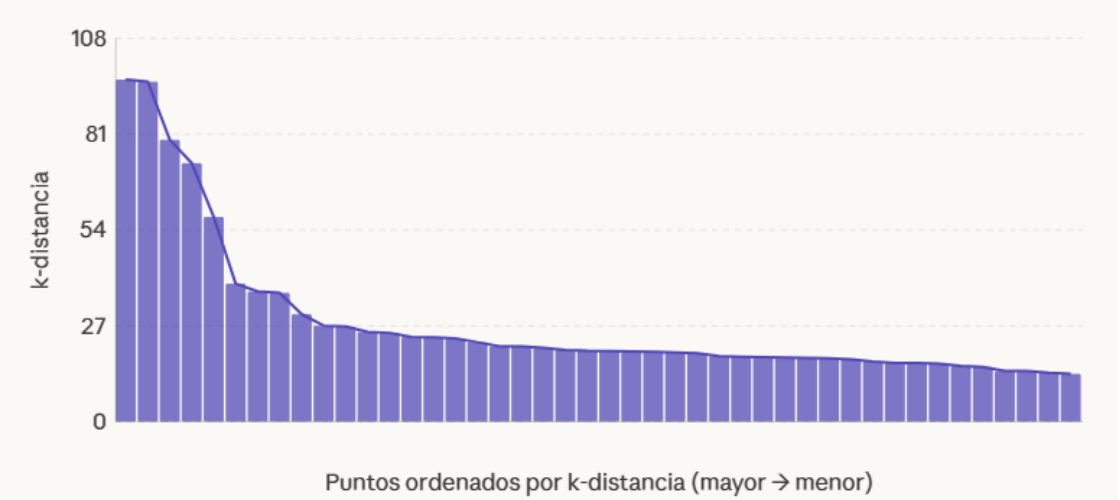
6. Repetir. Continúa hasta que todos los puntos en D hayan sido visitados.

7. Resultado final. Cada punto queda etiquetado con un número de clúster $\{1, 2, \ldots, k\}$ o con la etiqueta *Ruido*. El algoritmo devuelve ese vector de etiquetas y el número total de clústeres k encontrados.



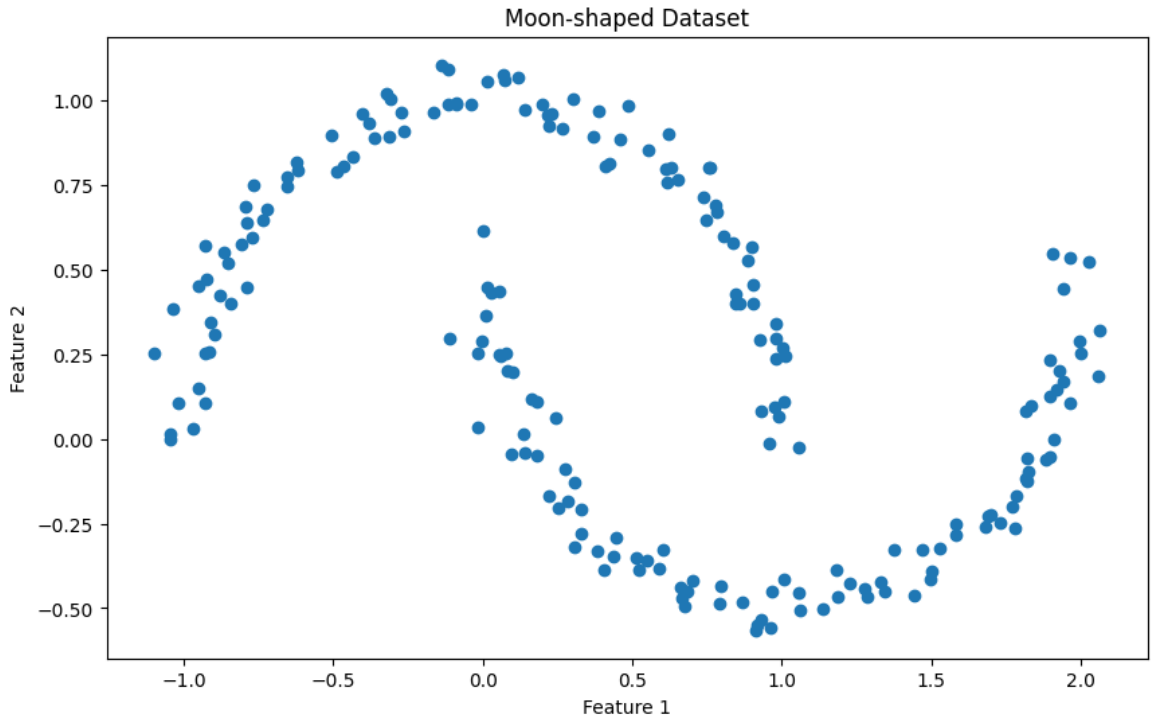
¿Cómo escoger los parámetros?

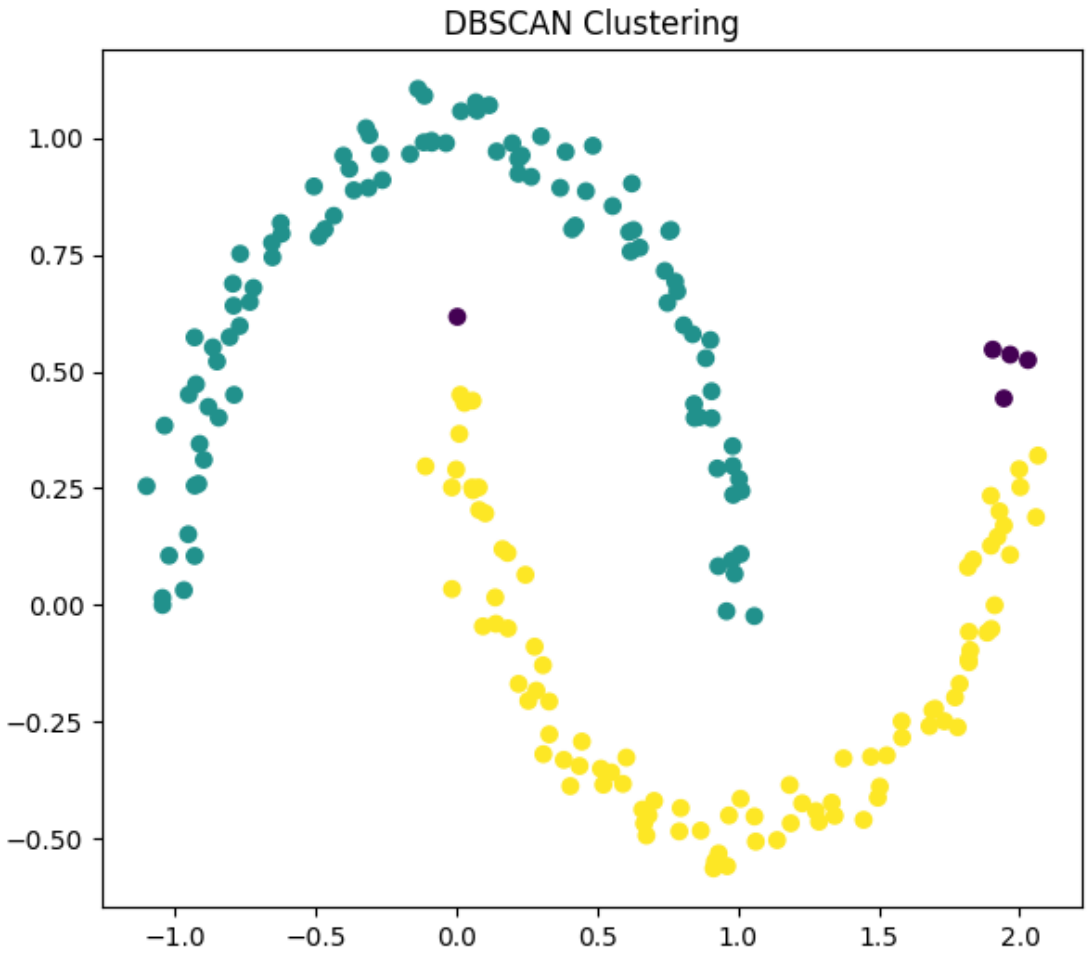
Parámetro	Intuición
MinPts	2 * numero de características Numero de características + 1
ε	Gráfica de k-distancia. Para cada punto se calcula la distancia al k-esimo vecino más cercano, ordenar distancias de mayor a menor y utilizar el valor en el codo.

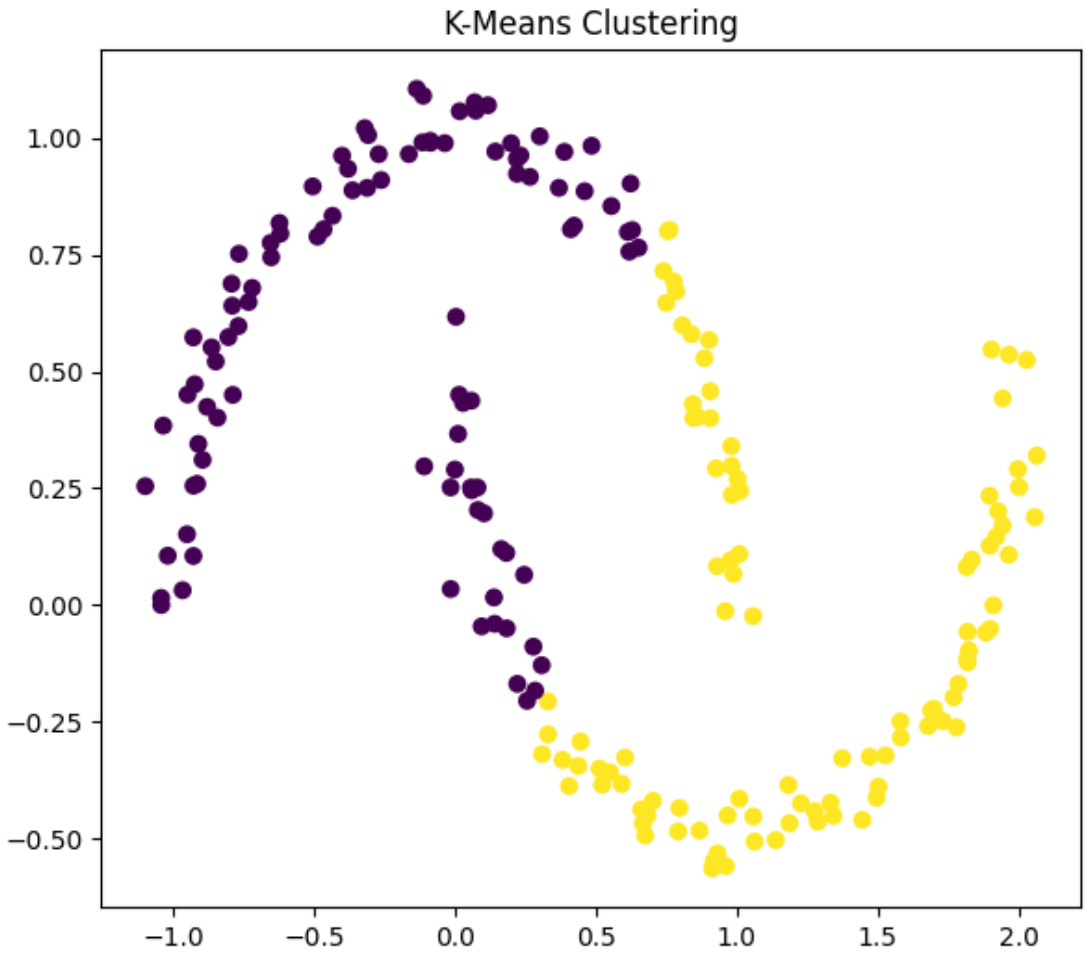




Resultados


Moon-shaped Dataset
Feature 2
Feature 1
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

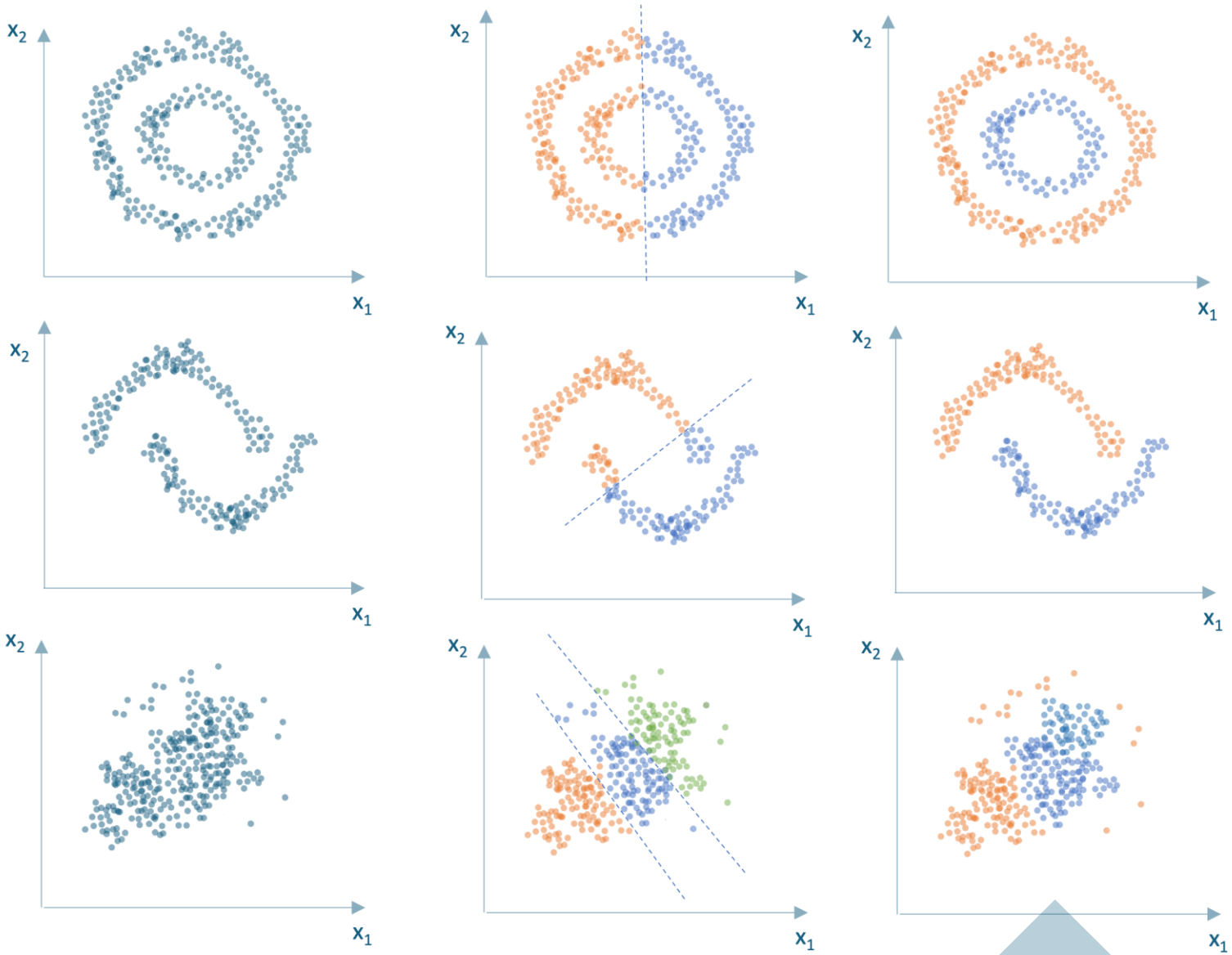

DBSCAN Clustering
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


K-Means Clustering
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

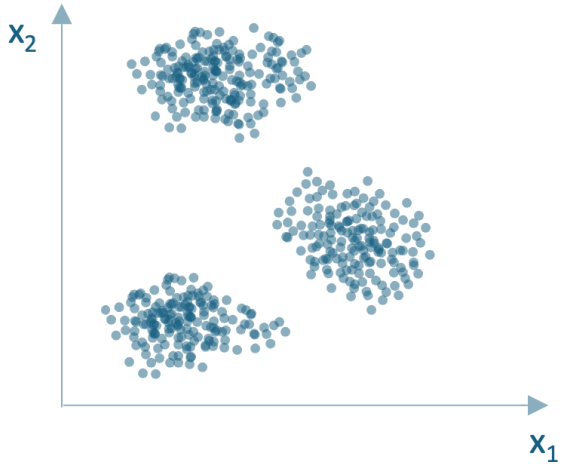
Muestra original

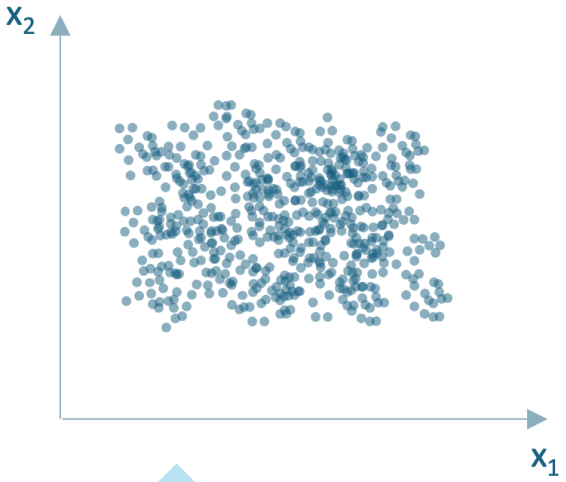
k-means

DBSCAN

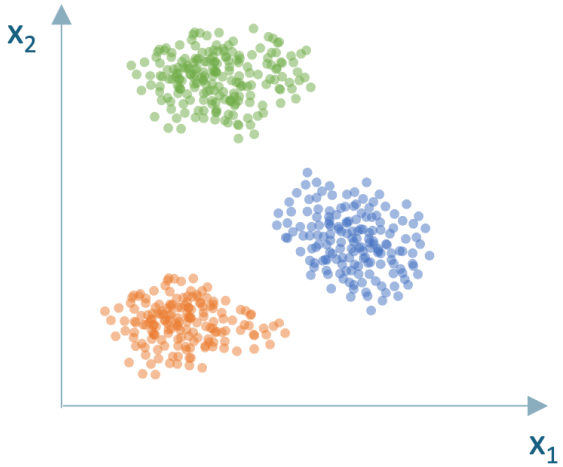


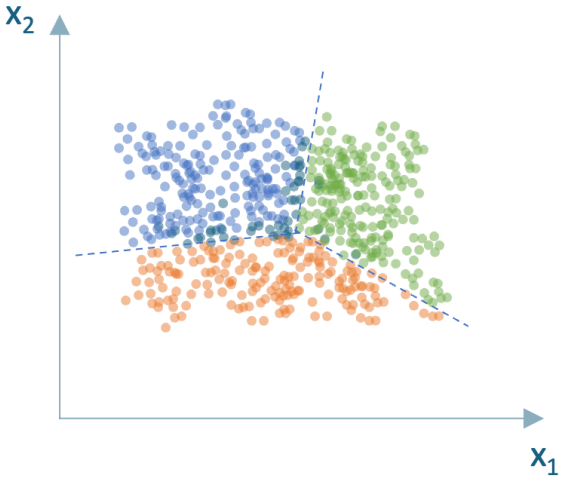
Muestra original



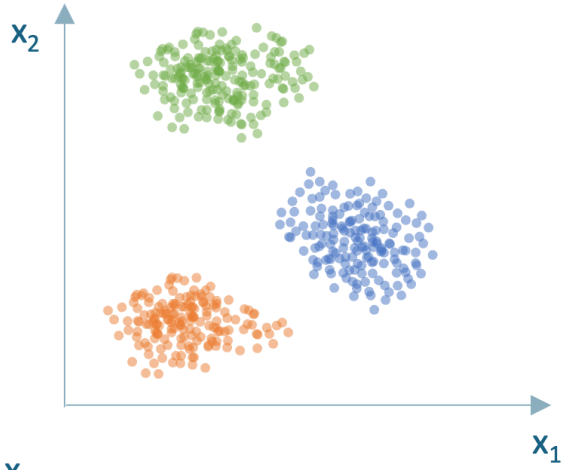


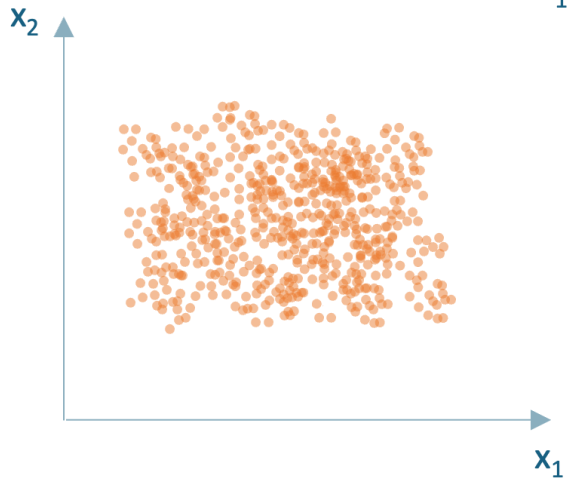
k-means

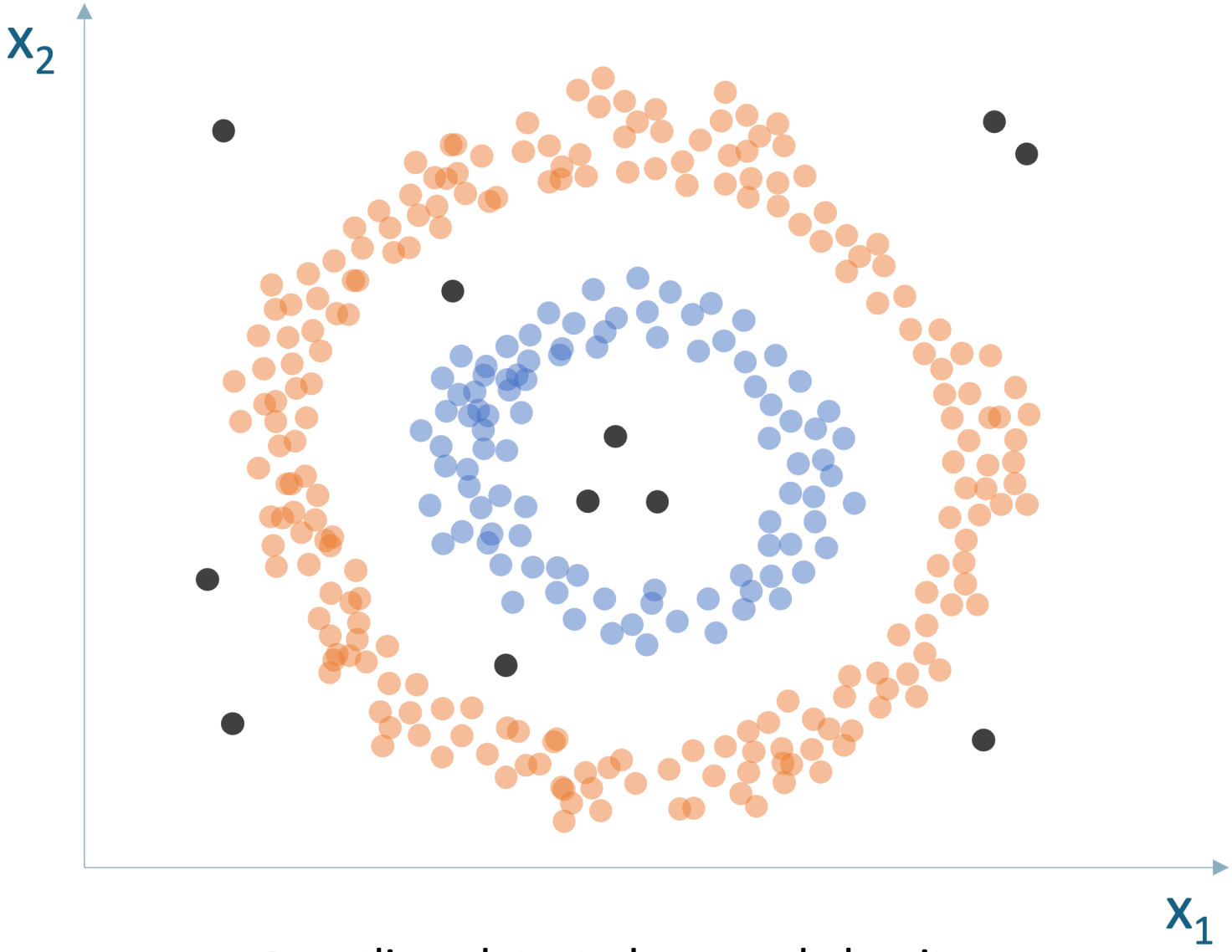




DBSCAN







● outliers detectados por el algoritmo

Ventajas

- No se debe de especificar el numero de clústeres
- Se detectan clústeres de forma arbitraria
- Se identifican outliers de forma natural
- Requiere solo de dos parámetros y no es susceptible al orden en que se analizan los puntos

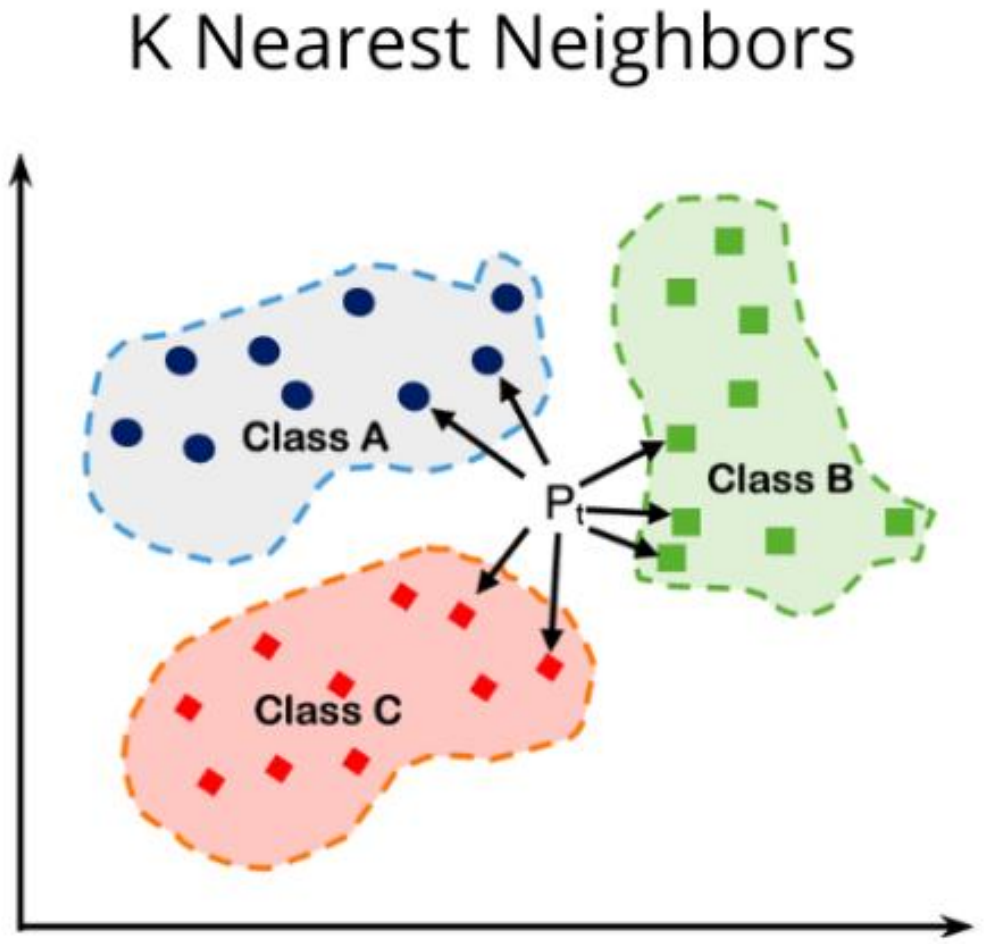
Desventajas

- Sensible a ε y MinPts
- Problemas con densidades variables
- Problemas con datos en alta dimensionalidad
- Si el conjunto de datos es muy grande puede volverse computacionalmente costoso

K-Nearest Neighbors (KNN)

Es un algoritmo de aprendizaje supervisado no paramétrico basado en instancias que emplea la proximidad para realizar predicciones.

La idea central es: **para clasificar un dato nuevo, mira quiénes son sus vecinos más cercanos y adopta la decisión de la mayoría**.



¿Cómo funciona?

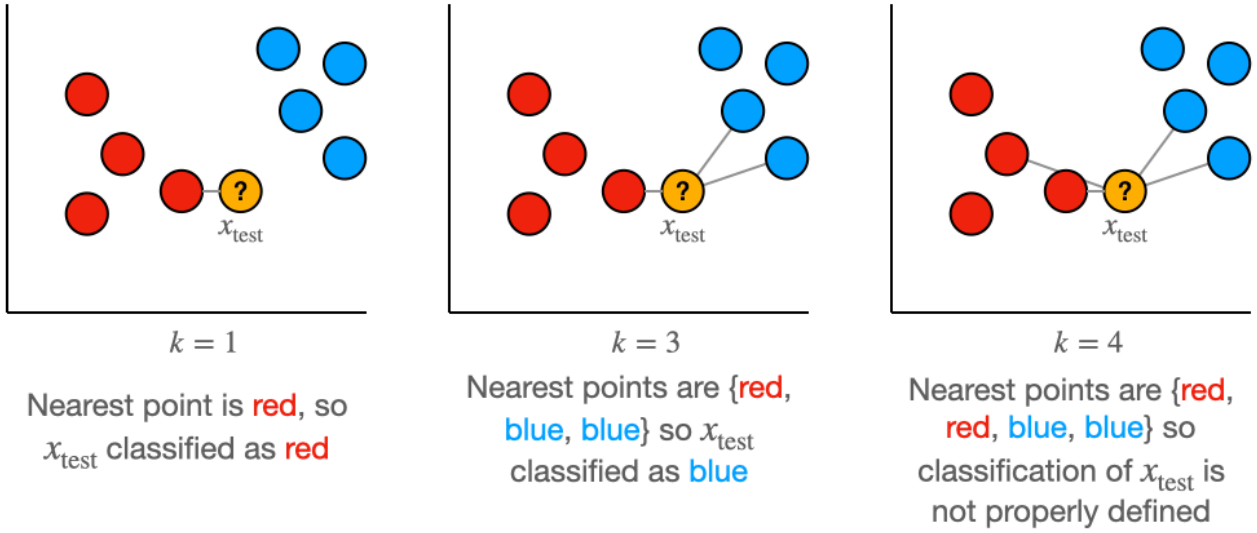
1. Almacenar todos los datos de entrenamiento.

Al recibir un nuevo punto x a clasificar:

2. Se calcula la distancia entre x y todos los puntos del dataset.

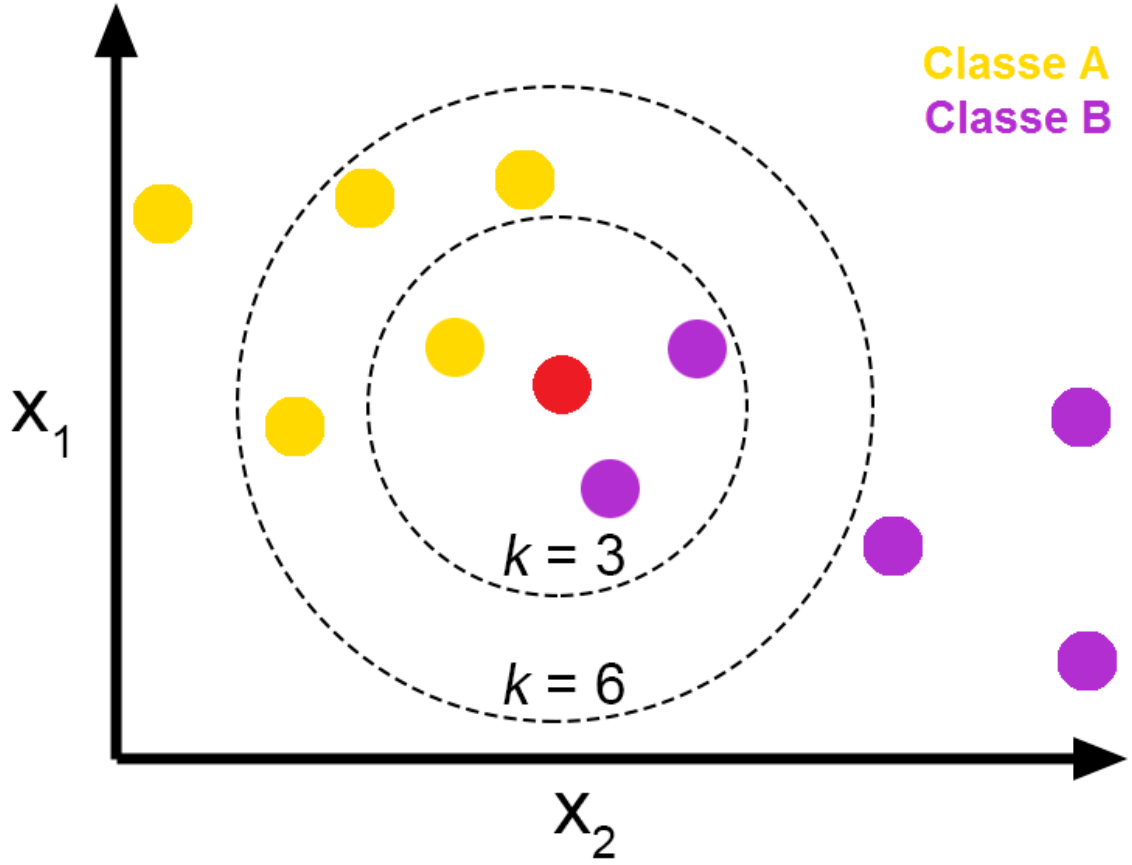
3. Se seleccionan los k vecinos más cercanos.

4. Se toma una decisión basada en la mayoría.



Seudocódigo

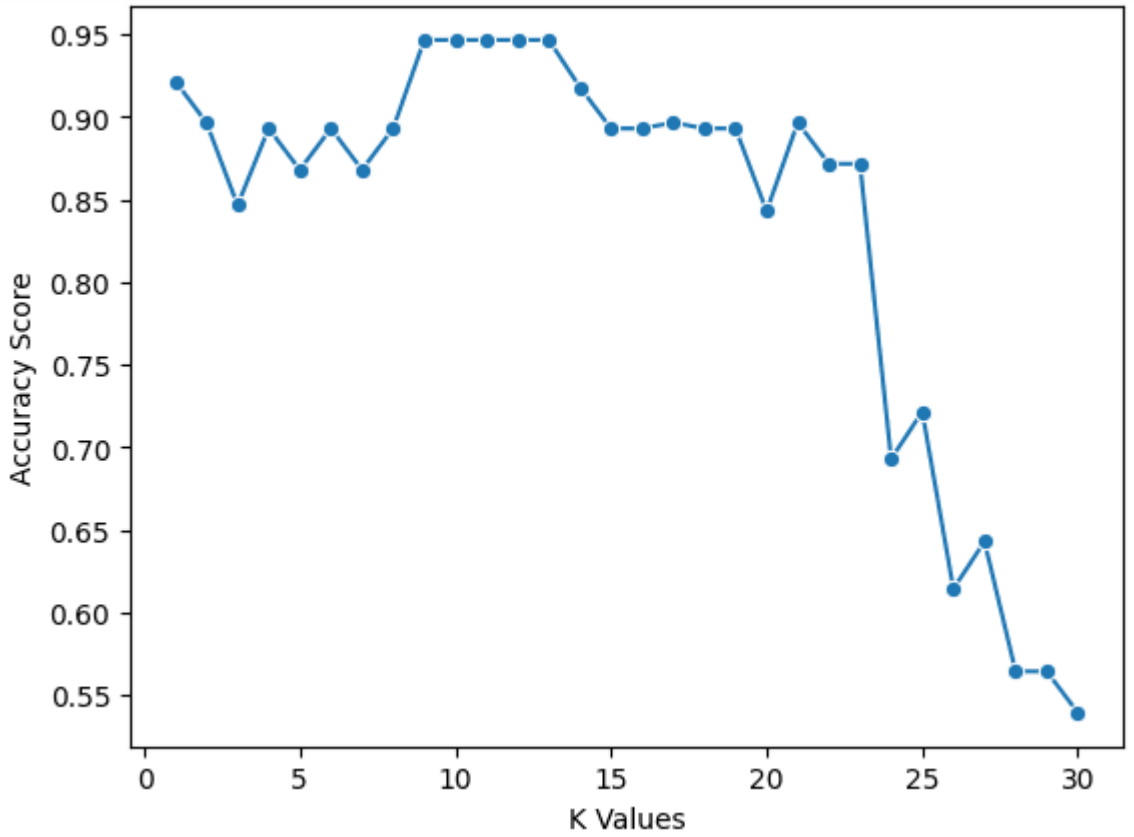
COMIENZO
 Entrada: $D = \{(\mathbf{x}_1, c_1), \ldots, (\mathbf{x}_N, c_N)\}$
  $\mathbf{x} = (x_1, \ldots, x_n)$ nuevo caso a clasificar
 PARA todo objeto ya clasificado (x_i, c_i)
  calcular $d_i = d(\mathbf{x}_i, \mathbf{x})$
 Ordenar $d_i (i = 1, \ldots, N)$ en orden ascendente
 Quedarnos con los K casos $D_{\mathbf{x}}^K$ ya clasificados
 más cercanos a $\mathbf{x}$
 Asignar a $\mathbf{x}$ la clase más frecuente en $D_{\mathbf{x}}^K$
FIN



También sirve para **regresión**

Elección de k

k pequeño	k grande
Muy sensible al ruido	Fronteras más suaves
Riesgo de sobreajuste	Riesgo de subajuste
Fronteras de decisión irregulares	Puede ignorar patrones locales



Funciones de distancia

$$d(X, Y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$$

- **Hamming**

El número de posiciones en las que los vectores correspondientes son diferentes.

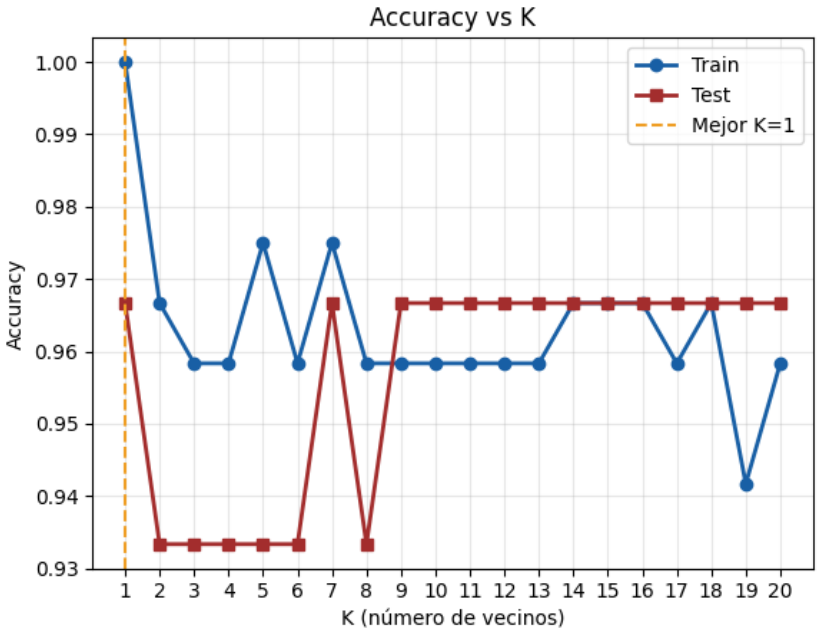
- **Manhattan**

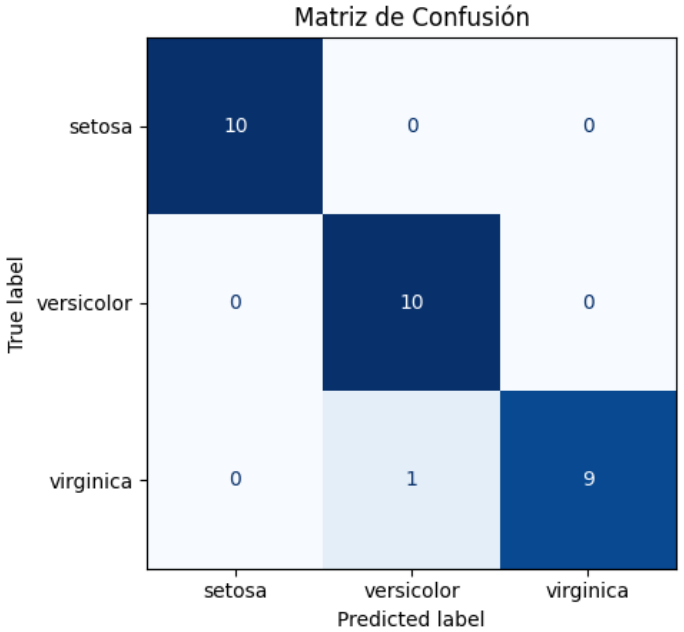
Calcula la distancia entre dos puntos sumando las diferencias absolutas de sus coordenadas.

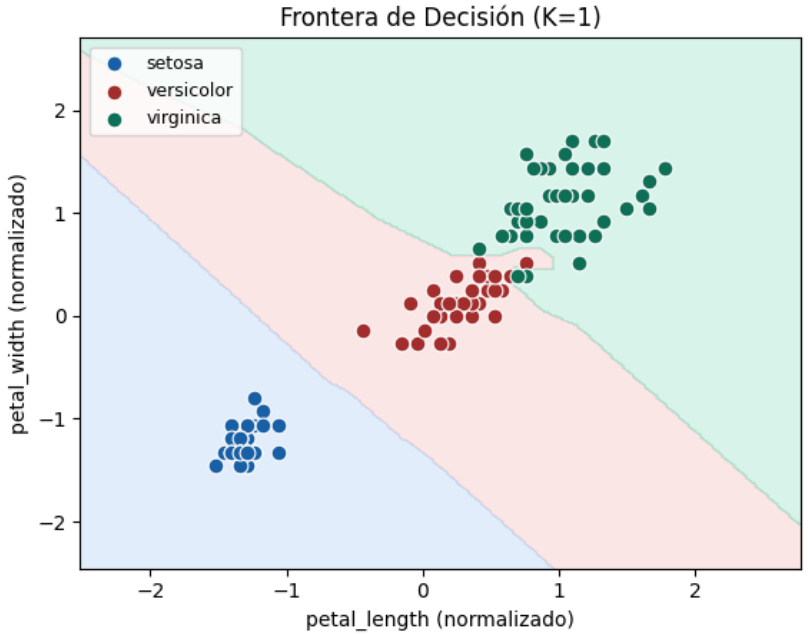
- **Euclidiana**

Es la distancia "en línea recta" entre dos puntos en un espacio euclidiano.

KNN con Dataset Iris







Consideraciones

Ventajas	Desventajas
Fácil de entender y de implementar	Es lento en predicción
No requiere entrenamiento ni asume una distribución de datos	Sensible a ruido y escalamiento
Funciona para clasificación y regresión	Sufre con datos de alta dimensionalidad
Pocos hiper parámetros	Propenso a sobreajuste

Aplicaciones

- Sistemas de recomendación
- Detección de anomalías
- Preprocesamiento: imputación de datos
- Sistemas generales de reconocimiento de patrones

Para la próxima clase...

Regresión Logística

